

Sistemas Operativos

Facultad de Informática
Universidad Complutense de Madrid

Módulo 1 - Introducción Sistemas Operativos

Outline

1 Introducción

2 Sistema de ficheros

3 Temario

4 Utilidades del SO



¿Necesitamos un sistema operativo?

- A estas alturas ya sabemos...
  - Programar en lenguajes de alto nivel
  - Programar en ensamblador
  - Gestionar la E/S a bajo nivel
- ¿Qué ha hecho el sistema operativo por nosotros todo este tiempo?
  - Veámoslo como un *proveedor de servicios*

Repasemos lo que ya sabemos...

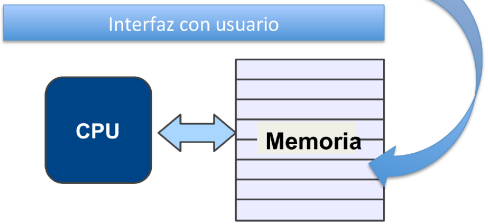
```
int main(void)
{
 int a, b, p;

 a = 5;
 b = 3;
 p = a * b;

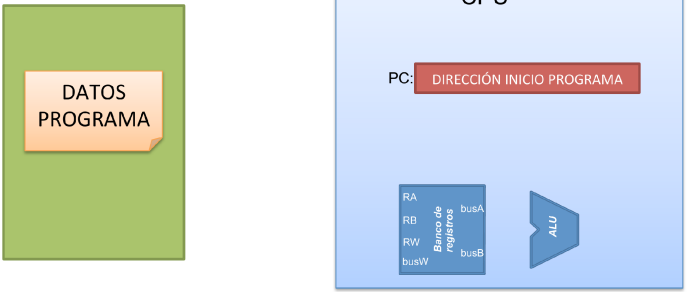
 …
 return 0;
}
```

```
main:
   leal   4(%esp), %ecx
   andl   $-16, %esp
   pushl  -4(%ecx)
   pushl  %ebp
   movl   %esp, %ebp
   pushl  %ecx
   subl   $16, %esp
   movl   $5, -16(%ebp)
   movl   $3, -12(%ebp)
……..
```

```
457f 464c 0101 0001 0000 0000 0000 0000
0002 0003 0001 0000 8280 0804 0034 0000
0df0 0000 0000 0000 0034 0020 0007 0028
0022 001f 0006 0000 0034 0000 8034 0804
8034 0804 00e0 0000 00e0 0000 0005 0000
0004 0000 0003 0000 0114 0000 8114 0804
8114 0804 0013 0000 0013 0000 0004 0000
0001 0000 0001 0000 0000 0000 8000 0804
8000 0804 046c 0000 046c 0000 0005 0000
1000 0000 0001 0000 046c 0000 946c 0804
946c 0804 0100 0000 0104 0000 0006 0000
1000 0000 0002 0000 0480 0000 9480 0804
9480 0804 00c8 0000 00c8 0000 0006 0000
0004 0000 0004 0000 0128 0000 8128 0804
8128 0804 0020 0000 0020 0000 0004 0000
```

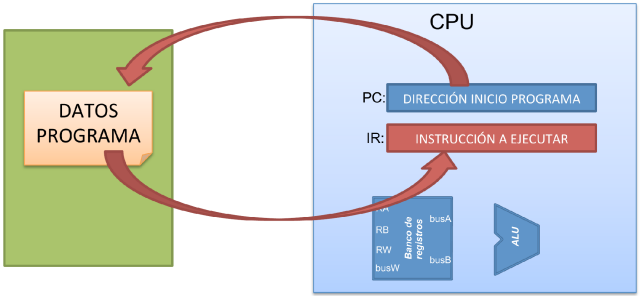


Ejecución instrucción a instrucción



La dirección de memoria de la siguiente instrucción se encuentra en el registro PC

Ejecución instrucción a instrucción

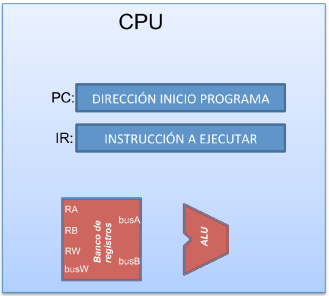


La CPU busca la instrucción que tiene que ejecutar

Qué tiene que hacer
Dónde están los datos
Dónde se escribe el resultado

Ejecución instrucción a instrucción

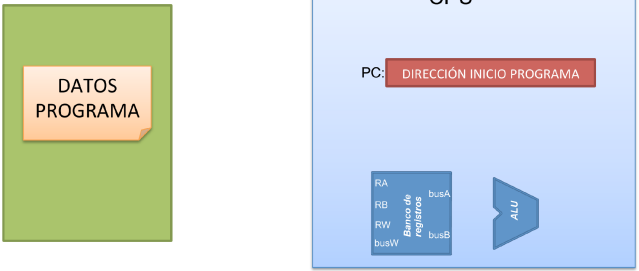




La CPU ejecuta la instrucción

Leer los datos
Realizar la operación
Escribir el resultado

Ejecución instrucción a instrucción



La CPU calcula automáticamente dónde se encuentra la siguiente instrucción del programa

Y ya está....
La CPU no da más de sí !!!



Pero entonces...

- ¿Cómo se lleva el fichero ejecutable a memoria?
  - ¡¡¡ Haciendo doble clic !!! Sí, claro.
- ¿A qué zona de la memoria se lleva?
- Una vez en memoria, ¿por qué/cómo empieza a ejecutarse mi código?
- El sistema operativo proporciona servicios para cargar un ejecutable en memoria y comenzar su ejecución



¿Y sólo puedo ejecutar un programa?

- Mi sistema operativo me permite ejecutar varios programas a la vez
  - Fácil, no hay más que repetir el proceso anterior *n* veces... ¿o no?
- Los programas deberán compartir múltiples recursos, empezando por la CPU
  - *"Es que mi ordenador tiene 4 cores"*
  - *"Y el mío, pero ahora mismo hay más de 100 programas ejecutándose a la vez"*



Gestión de procesos



- El sistema operativo ofrece la noción de **proceso**
  - Un proceso es un programa en ejecución
  - Ejecuto mi programa y, antes de que termine, lo vuelvo a ejecutar … ¿cuántos procesos se crean?
- A grandes rasgos, el sistema operativo reparte el uso de CPU entre los procesos
  - Planificador de procesos

El concepto de proceso

Proceso

El entorno de ejecución de un programa de usuario (el concepto se remonta a Multics, hacia 1960)

- El kernel guarda el estado de un proceso, que incluye:
  - Su espacio de direcciones de memoria
  - Los descriptores de fichero actualmente en uso
  - Los registros de la CPU
  - Una pila de memoria (una por hilo)
- La abstracción de proceso permite al SO:
  - Mejorar la eficiencia en el uso de recursos mediante multiprogramación (p. ej., un proceso usa la CPU mientras otro realiza una operación de E/S)
  - Repartir en el tiempo los recursos del sistema para que varios programas (y usuarios) puedan ejecutarse simultáneamente
  - Construir sistemas de tiempo real con una planificación determinista y aislada de las aplicaciones

Procesos, dinamismo, memoria...

- *"Tu ordenador puede tener 4 cores, pero seguro que sólo tiene una memoria..."*
  - Otro recurso que deben compartir los procesos
- Cuando decido ejecutar un programa (y se crea un proceso), ¿en qué posición de memoria escribo su código, datos...?
  - ¿Qué direcciones están libres/ocupadas?
  - ¿Y si el proceso A decide acceder a la memoria del proceso B?

Gestión de memoria

- El sistema operativo se hace cargo de...
  - Gestionar el espacio libre
  - Asignar memoria a los nuevos procesos
  - Evitar que un proceso acceda a zonas de memoria que pertenezcan a otro proceso
- Un momento, pero si hay más de 100 procesos ejecutándose a la vez... ¿caben todos en memoria?
  - Es probable que no
  - Más trabajo para el sistema operativo...

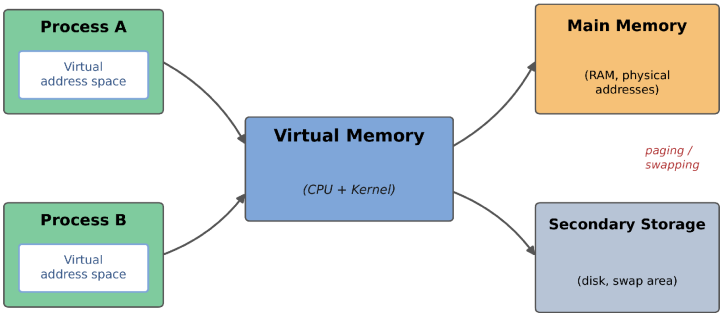
Memoria virtual (I)

Memoria virtual

Una abstracción que da a cada proceso (y al propio kernel) la ilusión de disponer de una memoria principal ilimitada y exclusiva

- Implementada conjuntamente por la CPU y el sistema operativo
- La memoria virtual permite:
  - La multitarea, ya que cada proceso trabaja dentro de su propio espacio de direcciones privado, sin conflictos
  - Sobreasignar memoria principal, respaldada por almacenamiento secundario
- Dos esquemas de implementación habituales:
  - **Swapping**: se mueven procesos completos entre memoria y disco
  - **Paginación**: solo se mueven pequeños segmentos de memoria (páginas, p. ej. 4 KB)
- En arquitecturas de 32 bits, el tamaño de dirección limita a un proceso a 4 GB de espacio de direcciones

Memoria virtual (II)



Comunicación y Sincronización

- El sistema operativo ofrece mecanismos de comunicación y sincronización
  - Entre procesos y entre hilos (ya veremos las diferencias)
- La proliferación de los procesadores multicore ha promovido la creación de aplicaciones multihilo
  - ¿Varios hilos de ejecución cooperando para acelerar los cálculos? No suena fácil
  - **Sincronización**: Los hilos tendrán que esperarse y notificar eventos
  - **Comunicación**: Los hilos pueden necesitar intercambiar información (p.ej., resultados intermedios)

Recapitulando...

El SO nos proporciona servicios para ...

- Ejecución de programas, mediante la creación de procesos
- Repartir el uso de la CPU y la memoria entre procesos
- Establecer mecanismos de comunicación y sincronización entre procesos (e hilos)
- Vale. Pero además de eso, ¿qué hace el sistema operativo por nosotros?

La información organizada, por favor

- Constantemente creamos y borramos ficheros y carpetas, pero... ¿qué es un fichero? ¿qué es una carpeta o directorio?
  - Son datos que están en el disco...¿¿y??
- Quiero abrir un fichero. ¿Qué hay que hacer?
  - Doble clic. Sí, claro...
  - ¿Cómo sé en qué sector del disco comienza ese fichero?

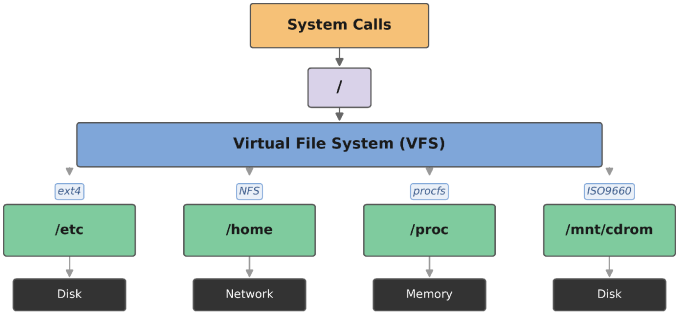
Organizando la información

- El disco es como la memoria, pero a lo bestia
- Mismos problemas que para la gestión de memoria:
  - ¿Qué zonas están libres/ocupadas?
  - Al crear un nuevo fichero, ¿dónde lo coloco?
  - Dado un fichero identificado por su ruta, ¿cómo lo encuentro en el disco?
  - Si un fichero es muy grande, ¿deben estar sus datos consecutivos en el disco?
  - Ese fichero me pertenece; mecanismos de privacidad/protección

La interfaz del sistema de ficheros (I)

- Un sistema de ficheros permite a los usuarios organizar los datos en ficheros y carpetas, dispuestos en forma de árbol
- El SO ofrece una interfaz común (POSIX) sobre estos ficheros: open, close, read, write...
- El kernel soporta varios tipos de sistemas de ficheros, todos accesibles a través de esa misma interfaz común
- Un sistema de ficheros sólo es accesible desde el árbol una vez que ha sido *montado*

La interfaz del sistema de ficheros (II)



A mounted filesystem is reachable from the tree, regardless of its type or backing store

Sobre discos, impresoras y ratones

- ¿Y si gestionar dispositivos de E/S fuera labor del programador de aplicaciones?
  - Esta labor puede ser un poco tediosa:
    - Interrupciones
    - Gestión de DMA
    - ...
- Definitivamente, que lo haga el SO:
  - Proporciona interfaces *friendly* para el acceso a los dispositivos
  - Permite arbitrar el acceso a los dispositivos de E/S por parte de distintos procesos
  - La funcionalidad ardua la realiza el driver
  - ¿Y cómo se comunica el driver con las aplicaciones?

Controladores de dispositivos

- El kernel interactúa con los dispositivos conectados al sistema (gestión y operaciones de E/S) a través de los controladores de dispositivo
  - Estos suelen admitir carga dinámica, sin necesidad de reiniciar
- Dos grandes tipos de dispositivo:
  - **Modo carácter (raw)**: sin buffer, acceso secuencial, en el tamaño que exija el dispositivo, hasta un único byte, p. ej., teclados, impresoras
  - **Modo bloque**: acceso aleatorio (por desplazamiento), en bloques de tamaño fijo (p. ej., 512 bytes). UNIX originalmente cacheaba los buffers de dispositivo para mejorar el rendimiento



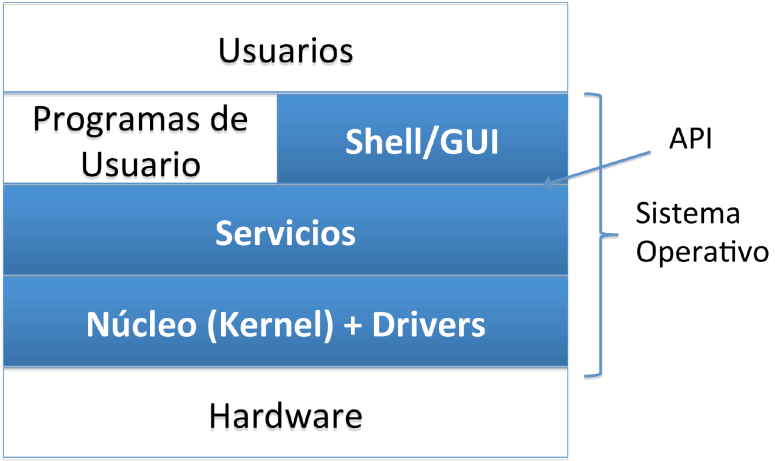
Contenido

- **Módulo 1** : Introducción a los sistemas operativos
- **Módulo 2** : Sistemas de ficheros
- **Módulo 3** : Procesos y sincronización
- **Módulo 4** : Gestión de entrada/salida
- **Módulo 5** : Gestión de memoria

Funciones del sistema operativo

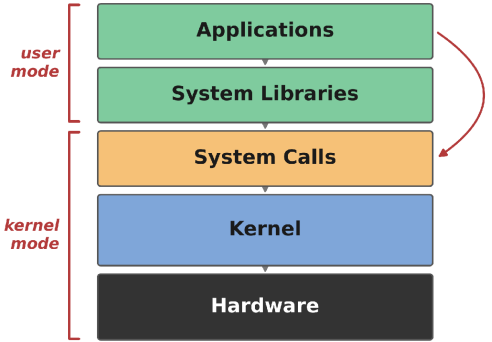
- **Gestión de recursos** (CPU, memoria, ...)
  - Asignación y recuperación de recursos
  - Soporte de usuario y protección entre usuarios
  - Contabilidad/monitorización
- **Servicios** (*máquina extendida*)
  - Ejecución de programas (procesos)
  - Gestión de los dispositivos de E/S
  - Operaciones sobre ficheros y directorios
  - Detección y tratamiento de errores
- **Interfaz** de usuario
  - *Shell* o intérprete de comandos
  - GUI (*Graphical User Interface*)

Niveles del sistema operativo



Kernel

El programa que gestiona todo el sistema: dispositivos hardware, memoria, planificación de CPU… (los detalles dependen del diseño del kernel)



Soporte hardware para el sistema operativo

- Muchas funciones del SO que requieren soporte HW:
  - Permitir la ejecución de varios procesos, posiblemente multiplexando el uso de CPU
  - Proteger el acceso a memoria entre procesos
  - Permitir que dos o más procesos compartan memoria
  - Permitir a los procesos que utilicen los dispositivos de E/S, garantizando que el SO arbitre el uso de los mismos
  - Evitar que los procesos de usuario corrompan el código y las estructuras de control del SO

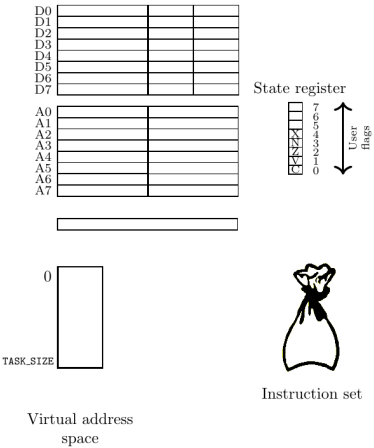
Tres mecanismos HW esenciales

1. Distintos modos de ejecución del procesador
2. Soporte HW para memoria virtual (MMU - Memory Management Unit)
3. Temporizador del sistema (interrupciones periódicas)

Modos de ejecución del procesador

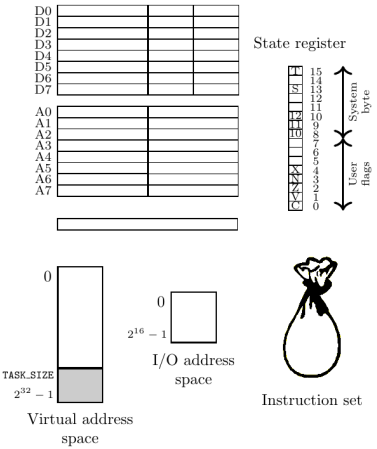
- Los procesadores actuales ofrecen un conjunto de modos de ejecución para proporcionar distintos niveles de acceso a los recursos de la máquina
- Cada modo de ejecución está caracterizado por:
  - Subconjunto de instrucciones del repertorio disponibles
  - Acceso al mapa de E/S
  - Acceso a los registros de soporte de gestión de memoria (config. MMU)
  - Permiso de cambio de modo (Bits del registro de estado)
- En GNU/Linux, el núcleo del SO se ejecuta en el modo menos restrictivo ("modo kernel") y los programas de usuario en el más restrictivo ("modo usuario")
- Las arquitecturas x86 (Intel y AMD) ofrecen 4 niveles de privilegio (*ring levels*): 0, 1, 2 y 3.
  - Modo usuario: ring level 3
  - Modo kernel: ring level 0

Modo usuario vs. modo kernel



- Modo Usuario

Modo usuario vs. modo kernel



- Modo Kernel

Excepción

- Un evento interno al procesador que desencadena un mecanismo de tratamiento Básicamente, esto consiste en:
  1. Detener la ejecución de instrucciones (el programa actual)
  2. Cambiar a modo privilegiado
  3. Guardar parte del estado arquitectónico
  4. Saltar a una dirección específica: el **vector de excepción**
     - Esa dirección debe contener una instrucción de salto a la rutina de tratamiento de la excepción (RTI o ISR)
     - Al volver de esa rutina se reanuda la ejecución donde se interrumpió, regresando a modo usuario

Interrupción

- Una excepción provocada por un evento externo
  - El tratamiento hardware es algo más complejo
  - El procesador no puede saber de antemano cuándo se producirá
  - El mecanismo de retorno también puede ser ligeramente distinto

Activación del sistema operativo

- Pero si el sistema operativo es simplemente software...
  - *si se empieza a ejecutar el reproductor de vídeo con una película, ¿cuándo/cómo puede ejecutarse código del sistema operativo?*
- El SO se comporta como un *servidor*: sus funciones se ejecutan en respuesta a eventos
  - Por invocación directa desde el programa de usuario (llamada al sistema)
    - Ej.: El reproductor solicita abrir y leer el fichero con la película
  - Por la llegada de una interrupción (un temporizador, por ejemplo)
  - Porque se produzca una excepción
    - Ej.: Hay un *bug* en el reproductor que desencadena una violación de segmento

Servicios del SO: llamadas al sistema

- Desde el punto de vista del programador, una llamada al sistema es como cualquier otra función
  - Las llamadas al sistema retornan un valor y pueden aceptar argumentos
- Pero en realidad, una llamada al sistema **NO** es una función convencional:
  - La llamada provoca la invocación de una función que no está en el espacio de direcciones del proceso sino del kernel del SO
    - Un proceso no tiene acceso a ese espacio de direcciones
    - No basta usar una instrucción de salto para transferir el control al SO
  - En SOs monolíticos (como GNU/Linux), la función debe ejecutarse completamente en modo kernel (modo privilegiado)
- La invocación de una llamada al sistema desde un proceso de usuario fuerza un cambio de modo usuario a modo kernel
  - Instrucción especial del repertorio (TRAP)

Ejemplos de llamadas al sistema

- El kernel se ejecuta en el modo privilegiado de la CPU, con acceso sin restricciones a los recursos del sistema; el resto de aplicaciones se ejecutan en modo usuario, con acceso restringido
- Las llamadas al sistema son el mecanismo que permite a las aplicaciones realizar acciones privilegiadas
  - Se ofrecen interfaces sencillas para acceder a esta funcionalidad (p. ej., libc)
- Algunos ejemplos: write(2), read(2), open(2), stat(2), connect(2)...
- La herramienta strace(1) muestra qué llamadas al sistema realiza un programa

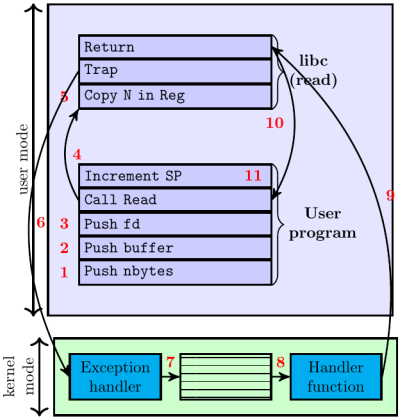
Ejemplo de llamada al sistema

```
int main()
{
        char buffer[128];
        int count;
        int nbytes=128;
        int fd;

        ... Open the file  ...

        count=read(fd,buffer,nbytes);

        ...
}
```



Un ejemplo concreto: strace (I)

En x86-64, una llamada a sistema se dispara cuando se produce una interrupción software, utilizando la instrucción especial `syscall`

Por ejemplo:

helloworld.cpp

```
#include <iostream>

int main(){
   std::cout << "HOLA MUNDO!n";
   exit(0);
}
```

Un ejemplo concreto: strace (I)

- El ensamblador para la llamada `write(2)`:

assembler.c

```
 ssize*t write(int fd, const void *buf, size*t count)

mov    rax, 1     ; rax holds the call number: 1 = write
mov    rdi, 1     ; fd: 1, standard output
mov    rsi, msg   ; buf: pointer to the string buffer
mov    rdx, msglen ; count: length of the string
syscall           ; system call (into the kernel)
```

Un ejemplo concreto: strace (II)

- Lo que muestra `strace` para el programa de ejemplo:

Terminal #1

```
user@lab:~$ strace ./helloworld**
...
write(1, "HOLA MUNDO!\n", 12)           = 12
exit_group(0)                           = ?
+++ exited with 0 +++
```

Estándar POSIX

- Interfaz estándar de sistemas operativos del IEEE
- Objetivo: portabilidad de las aplicaciones entre diferentes plataformas y sistemas operativos
  - POSIX: *Portable Operating System Interface for uniX*
- NO es una implementación. Sólo define una interfaz
- Diferentes estándares
  - 1003.1.: Servicios básicos del SO
  - 1003.1a: Extensiones a los servicios básicos
  - 1003.1b: Extensiones de tiempo real
  - 1003.1c: Extensiones de hilos
  - 1003.2: Shell y utilidades
  - 1003.2b: Utilidades adicionales

Características de POSIX

- Nombres de funciones cortos y en letras minúsculas
  - `fork()`
  - `read()`
  - `close()`
- Las funciones normalmente devuelven 0 en caso de éxito o –1 en caso de error
  - Código de error se almacena en la variable global errno
- Recursos gestionados por el sistema operativo se referencian mediante *descriptores*

Activación del sistema operativo

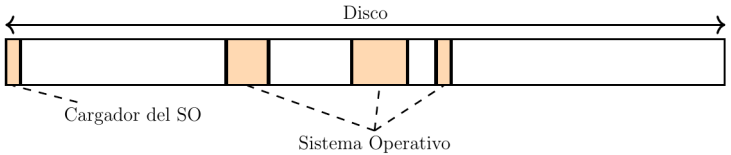
- El sistema operativo se activa también periódicamente
  - Interrupciones generadas por el temporizador del sistema
  - Cada interrupción provoca una transición de modo usuario a modo kernel
- Tareas periódicas del SO
  - Planificación: CPU accounting, expropiaciones de procesos/hilos, equilibrado de carga
  - Gestión del almacenamiento intermedio (escrituras a disco desde la cache de bloques)
  - Tareas periódicas asociadas a los protocolos de red
  - ...

Planificación en la práctica

- En un sistema de tiempo compartido, el planificador es responsable de asignar procesos (hilos) a las CPUs del sistema
- Objetivos del planificador:
  - Repartir los ciclos de CPU entre los procesos activos
  - Establecer una prioridad (dinámica) entre procesos. Los hilos del kernel tienen prioridad sobre los hilos de usuario
- Caracterización de procesos:
  - *CPU-bound**: realiza principalmente cómputo
  - **I/O-bound**: realiza principalmente operaciones de E/S (p. ej., red o disco)
- Ejemplo en UNIX: el planificador aumenta la prioridad de las aplicaciones I/O-bound, para intentar reducir su latencia. Para cada proceso:
  - Tasa de CPU = tiempo de CPU / tiempo real
  - A los procesos con una tasa de CPU alta se les reduce la prioridad

¿Y cómo empieza todo?: Arranque del SO

- Los datos y el código del SO se almacenan en disco



- El computador incluye una memoria ROM (no volátil) que contiene un programa (iniciador ROM) encargado de:
  - Transferir a memoria el cargador del SO
  - Ejecutar el cargador del SO (PC <- Dirección de la primera instrucción)

Arranque del sistema

- **1. Iniciador ROM**
  - Test del HW
  - Cargar en memoria el cargador del SO y transferir control
- **2. Cargador del SO**
  - Cargar en memoria los distintos componentes del SO
- **3. Sistema operativo**
  - Test del sistema de ficheros
  - Creación de estructuras de datos internas (planificación, gestión de memoria, ...)
  - Paso, si procede, a modo Memoria Virtual
  - Cargar controladores de dispositivos (*drivers*)
  - Habilitar interrupciones
  - Crear proceso de *login*